

Canonical Polyadic Decomposition and applications

Sottotitolo

by

Alberto Defendi (with the help of many others!)

A document submitted in partial fulfillment of the requirements for the degree of

Technical Report

at

MISKATONIC UNIVERSITY

ABSTRACT

Scientific documents often use L^AT_EX for typesetting. While numerous packages and templates exist, it makes sense to create a new one. Just because.

INDICE

1	APPLICAZIONI DELLA CP ALLA COMPLESSITÀ ARITMETICA	1
1.1	L'esponente della moltiplicazione tra matrici	1
1.2	Mappe multilineari e moltiplicazione tra matrici	2
1.3	Decomposizione low-rank di forme bilineari	4
1.4	Rappresentazione del prodotto tra matrici	6
1.5	Alcune stime sull'esponente	6
1.6	Rango come misura della complessità	8
1.7	Algoritmi di moltiplicazione veloce	9
1.8	Algoritmi APA	10
1.9	Errore algoritmico	12
2	APPLICAZIONI DELLA CP ALLA ??	13
2.1	Fluorescenza spettroscopica	13
2.2	Cocktail party problem	13
3	APPENDICE	15
3.1	Il master method	15
	ACRONIMI	17
	GLOSSARIO	19
	BIBLIOGRAFIA	21

1 APPLICAZIONI DELLA CP ALLA COMPLESSITÀ ARITMETICA

1.1 L'ESPONENTE DELLA MOLTIPLICAZIONE TRA MATRICI

In questo capitolo useremo il seguente modello computazionale per il calcolo della complessità, per calcolare costi computazionali su campi infiniti, e semplificare il calcolo dei costi eliminando branching (e.g. `if-then-else` o `while`).

Definizione 1.1.1 (Straight line program). Uno *straight line program* (o *circuito algebrico*) su un campo $\mathbb{K}$ è una sequenza ordinata di istruzioni della forma

$$(\mathbf{op}; i, j) \text{ oppure } (\lambda; i)$$

dove $\mathbf{op} \in \{+, -, *, /\}$ è una delle quattro operazioni aritmetiche fondamentali (binarie) e $\lambda \in \mathbb{K}$. Il primo tipo di istruzioni descrive l'esecuzione dell'operazione aritmetica $\mathbf{op}$ fra gli i -esimi e j -esimi risultati intermedi, mentre il secondo tipo descrive la moltiplicazione dell' i -esimo risultato intermedio con lo scalare λ .

Definizione 1.1.2 (Funzione di complessità aritmetica).

Indichiamo con L^{tot} la funzione che calcola la complessità aritmetica totale (addizioni e moltiplicazioni) per il calcolo di una funzione φ . Fissato un campo $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$, e delle matrici $X, Y \in \mathbb{R}^{n \times n}$, denotiamo con

$$(1.1) \quad M_{\mathbb{K}}(n) := L^{\text{tot}}(XY) = L^{\text{tot}}\left(\left\{\sum_{j=1}^n x_{ij}y_{jl} \mid i, l \in \{1, \dots, n\}\right\}\right)$$

il numero totale di moltiplicazioni aritmetiche sufficienti a calcolare la moltiplicazione di due matrici $n \times n$. Determinare una formula per $M_{\mathbb{K}}(n)$ è lontano dagli strumenti attualmente disponibili. Invece, il problema di approssimare la crescita asintotica della funzione $n \mapsto M_{\mathbb{K}}(n)$ è fattibile; definiamo

Controlla che non ci sia ambiguità nel riferirti a mappe/forme bilineari:

$\psi : U \times V \rightarrow W$ è una forma bilineare se $W = \mathbb{K}$, altrimenti chiamala mappa bilineare

Vedi PAG 105 [5] se raffinare

la definizione di L^{tot} è molto semplificata dal libro "Algebraic Complexity theory" [5]

Definizione 1.1.3. L'esponente $\omega_{\mathbb{K}}$ di una moltiplicazione matriciale è il numero

$$\omega_{\mathbb{K}} := \inf_n \{ \tau \in \mathbb{R} \mid \text{la moltiplicazione tra due matrici in } \mathbb{K}^{n \times n} \text{ ha costo } O(n^\tau) \text{ operazioni aritmetiche} \}.$$

In altre parole, $\omega_{\mathbb{K}} = \inf \{ \tau \in \mathbb{R} \mid M_{\mathbb{K}}(n) = O(n^\tau) \}$.

Osservazione 1.1.4. Dalla notazione $M_{\mathbb{K}}(h)$ e $\omega_{\mathbb{K}}$ è dovuta alla dipendenza dal campo $\mathbb{K}$. Poiché il seguente risultato avanzato afferma che l'esponente dipende solo dalla caratteristica del campo, e tratteremo solo $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$, spesso scriveremo ω per $\omega_{\mathbb{K}}$.

Teorema 1.1.5 (Schönhage [14]). *L'esponente della moltiplicazione tra matrici è invariante rispetto alle estensioni scalari: se $\mathbb{K} \subset \mathbb{L}$ è un'estensione di campo, allora*

$$\omega_{\mathbb{K}} = \omega_{\mathbb{L}}$$

Osserviamo che $M_{\mathbb{K}}(n) \leq n^2(2n-1)$. Siccome il costo computazionale dev'essere almeno pari al numero di entrate della matrice, vale $M_{\mathbb{K}}(n) \geq n^2$, da cui $\omega \in [2, 3]$. Trovare ω è un problema centrale nella teoria della complessità, e si congettura che $\omega = 2$, cioè che il costo della moltiplicazioni possa essere asintoticamente pari a quello delle moltiplicazioni. L'algoritmo naive ha $\omega = 3$, mentre il primo miglioramento, dovuto all'algoritmo di Strassen [17] mostra che $\omega \leq \log_2(7) < 2.81$. Il record attuale è $\omega \leq 2.371339$ [1]. Dopo Strassen, nel 1978 i miglioramenti significativi sono stati $\omega \leq 2.7962$ ad opera di [13], poi $\omega \leq 2.7799$ [2] nel 1979, poi $\omega \leq 2.55$ [14], poi $\omega \leq 2.48$ nel 1987 [16], e poi $\omega \leq 2.38$ [6] nel 1989, che potrebbe aver fatto pensare che nel 1990 si sarebbe trovato il bound. Dopo tale miglioramento, l'approssimazione non è più scesa sotto $\omega < 2.373$ [1, 10, 15, 18]. In futuro, potremmo aspettarci ulteriori miglioramenti, anche grazie all'avvento dell'IA nella ricerca di algoritmi [9].

Osservazione 1.1.6. Nella maggior parte delle applicazioni, l'algoritmo di Strassen è quello ottimale, si veda la discussione in [8]. Di recente, il problema di stimare ω e trovare il rango di tensori è diventato un problema d'interesse della branca della teoria della complessità algebrica e della geometria algebrica. Si veda ad esempio [12].

1.2 MAPPE MULTILINEARI E MOLTIPLICAZIONE TRA MATRICI

Fix conflicting notation!

Siano M, N, P spazi vettoriali di dimensione finita, di dimensione m, n, p rispettivamente. Una forma bilineare è una mappa $f : M \times N \rightarrow P$ lineare in ogni

1.2 Mappe multilineari e moltiplicazione tra matrici

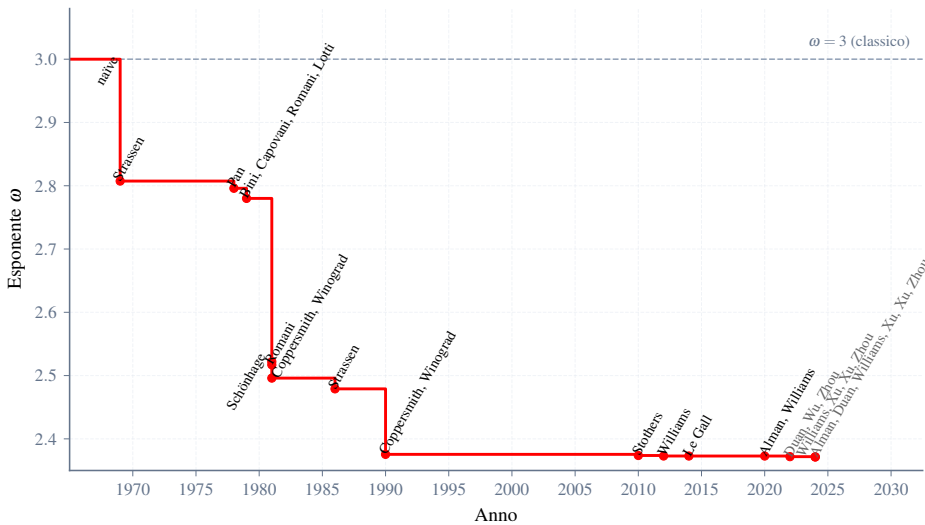


Figura 1.1: Miglioramenti del limite superiore dell'esponente della moltiplicazione matriciale ω dal 1969 ad oggi. La linea tratteggiata rappresenta il limite superiore $\omega \leq 3$.

componente, in altre parole tale che $f(m, \cdot)$ e $f(\cdot, n)$ sono lineari per ogni $m \in M$ e $n \in N$. In maniera analoga si definiscono le mappe trilineari $f : M_1 \times M_2 \times M_3 \rightarrow P$.

Definizione 1.2.1 (Moltiplicazione matriciale). Fissati degli interi positivi M, N, P , la moltiplicazione tra due matrici $M \times N$ e $N \times P$ è una mappa bilineare

$$\begin{aligned} \langle M, N, P \rangle: \mathbb{C}^{M \times N} \times \mathbb{C}^{N \times P} &\rightarrow \mathbb{C}^{M \times P} \\ (A, B) &\mapsto C = AB. \end{aligned}$$

Spesso ci riferiremo a $\langle M, N, P \rangle$ chiamandolo "algoritmo", ad esempio nel contesto degli algoritmi di moltiplicazione veloce.

Osservazione 1.2.2. Possiamo vedere la moltiplicazione tra matrici anche come una mappa trilineare

$$\begin{aligned} \mathbb{C}^{M \times N} \times \mathbb{C}^{N \times P} \times \mathbb{C}^{M \times P} &\rightarrow \mathbb{C} \\ (A, B, C) &\mapsto \text{trace}(ABC) \end{aligned}$$

Conoscere il rango CP di tensori che rappresentano applicazioni multilineari può rivelare informazioni sulla complessità asintotica e fornire algoritmi con complessità ottimale per la loro valutazione. Proviamo a formulare il problema.

Sfruttando l'isomorfismo con lo spazio vettoriale standard, possiamo rappresentare una forma bilineare $f: X \times Y \rightarrow Z$ come un tensore nel seguente modo: fissato $k \in \{1, \dots, p\}$ e considerata $f_k: X \times Y \rightarrow \mathbb{R}$, esiste una matrice $M \in \mathbb{R}^{m \times n}$ tale

giustifica se
necessario

che $f_k(x, y) = x^\top M y$ per ogni $(x, y) \in \mathbb{R}^m \times \mathbb{R}^n$. Scrivendo ogni componente di $f = \begin{bmatrix} f_1 \\ \vdots \\ f_p \end{bmatrix}$ in questa forma, otteniamo un tensore $\mathcal{T} \in \mathbb{R}^{m \times n \times p}$ tale che le sue slice frontali rappresentino ognuna delle p forme bilineari f_k , ossia che

$$(1.2) \quad f_k(x, y) = x^\top \mathcal{T}_{:, :, k} y = \sum_{i=1}^m \sum_{j=1}^n \mathcal{T}_{i, j, k} x_i y_j, \text{ per ogni } k = 1, \dots, p.$$

In forma tensoriale,

$$(1.3) \quad f(x, y) = \begin{bmatrix} f_1(x, y) \\ \vdots \\ f_p(x, y) \end{bmatrix} = \begin{bmatrix} x^\top \mathcal{T}_{:, :, 1} y \\ \vdots \\ x^\top \mathcal{T}_{:, :, p} y \end{bmatrix} = \mathcal{T} \times_1 x^\top \times_2 y^\top.$$

Osservazione 1.2.3. Questa procedura si estende (passando alle coordinate) anche al caso in cui le entrate di x e y siano oggetti algebrici più complessi di scalari, ad esempio vettori o matrici.

1.3 DECOMPOSIZIONE LOW-RANK DI FORME BILINEARI

Spostare davanti alla CP per alleggerire (già scritta)

Nel calcolo della complessità, le decomposizioni low-rank possono ridurre notevolmente le operazioni aritmetiche sulle matrici. Se ad esempio $M \in \mathbb{R}^{m \times n}$ è una matrice di rango uno, possiamo scrivere $M = u \circ v = uv^\top$ per certi $u \in \mathbb{R}^m$ e $v \in \mathbb{R}^n$; valutare $Mx = (uv^\top)x = u(v^\top x)$ ha costo $O(n)$, contro il costo $O(mn)$ del prodotto classico matrice vettore. In generale, se M ha rango R , possiamo scriverla come somma di R matrici di rango uno ed ottenere un costo $O(Rn)$ per Mx . È possibile riadattare quest'idea per forme bilineari, come vedremo ora.

aggiungere thm?

Calcolare la valutazione di una forma bilineare f come nell'Equazione 1.2 ha un costo $O(mnp)$, data dal numero di moltiplicazioni necessarie fra gli elementi di x e y . Queste sono dette *active multiplications* e corrispondono al numero di elementi non nulli di $\mathcal{T}$. Ci aspettiamo che la complessità di valutare $f(x, y)$ sia correlata al costo delle active multiplications. Cerchiamo di minimizzarne il numero cercando fattorizzazioni CP di $\mathcal{T}$.

Definizione 1.3.1 (Bilinear algorithm). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare

tra $\mathbb{K}$ -spazi vettoriali. Per ogni $i \in \{1, \dots, r\}$, siano $f_i \in U^*, g_i \in V^*, w_i \in W$ tali che

$$\varphi(u, v) = \sum_{i=1}^r f_i(u)g_i(v)w_i,$$

per ogni $u \in U, v \in V$. Allora la r -upla $(f_1, g_1, w_1; \dots; f_r, g_r, w_r)$ viene detta *bilinear computation (algorithm) di lunghezza r per φ* , e la lunghezza di più breve algoritmo di calcolo per φ è chiamata *bilinear complexity* o rango di φ , che indichiamo ancora con $\mathbf{R}(\varphi)$.

La prossima definizione assicura che il rango di una forma bilineare è dato dal rango CP.

Proposizione 1.3.2. *Fissata una forma bilineare f , e detto T il tensore che la rappresenta. Allora, dalla fattorizzazione CP di un tensore T si ottiene una bilinear computation.*

Dimostrazione. Data una decomposizione $\mathcal{T} = \llbracket U, V, W \rrbracket$ di rango r , dalla Definizione ?? abbiamo che $\mathcal{T} = \sum_{l=1}^r u_l \circ v_l \circ w_l$, o in componenti, $\mathcal{T}(i, j, k) = \sum_{l=1}^r u_{il}v_{jl}w_{kl}$. Inserendo questa relazione nella Equazione 1.2 troviamo,

$$\begin{aligned} (1.4) \quad f_k(x, y) &= \left(\left(\sum_{l=1}^r u_l \circ v_l \circ w_l \right) \times_1 x^\top \times_2 y^\top \right)_k = \left(\sum_{l=1}^r (u_l \circ v_l \circ w_l) \times_1 x^\top \times_2 y^\top \right)_k \\ &= \sum_{i=1}^m \sum_{j=1}^n \sum_{l=1}^r u_{il}v_{jl}w_{kl}x_iy_j = \sum_{l=1}^r \left(\sum_{i=1}^m u_{il}x_i \right) \cdot \left(\sum_{j=1}^n v_{jl}y_j \right) w_{kl} \\ &= \sum_{l=1}^r (u_l^\top x) \cdot (v_l^\top y) w_{kl}, \end{aligned}$$

per ogni $k = 1, \dots, p$. Mettendo insieme le Equazioni 1.3 e 1.4 troviamo

$$(1.5) \quad f(x, y) = \sum_{l=1}^r (u_l^\top x) \cdot (v_l^\top y) w_l.$$

□

Quindi valutare f richiede esattamente $r = \mathbf{R}(\mathcal{T})$ active multiplications (e $O(r)$ addizioni), che sopra evidenziamo con $(\cdot)$. In particolare, vale il seguente risultato algebrico:

Proposizione 1.3.3 (Si veda [5], Cap. 14). *Siano U, V e W dei $\mathbb{K}$ -spazi vettoriali. Allora esiste un unico isomorfismo $U^* \otimes V^* \otimes W \rightarrow \{\varphi : U \times V \rightarrow W, \text{ bilineare}\}$ che manda $f \otimes g \otimes w$ nella mappa bilineare $(u, v) \mapsto f(u)g(v)w$.*

Definizione 1.3.4 (Tensore strutturale). L'unico tensore $T \in U^* \otimes V^* \otimes W$ associato alla mappa bilineare $\varphi : U \times V \rightarrow W$ è detto *tensore strutturale* di φ .

1.4 RAPPRESENTAZIONE DEL PRODOTTO TRA MATRICI

Consideriamo due matrici $A \in \mathbb{R}^{M \times N}$ e $B \in \mathbb{R}^{N \times P}$ e il loro prodotto $C \in \mathbb{R}^{M \times P}$. Ponendo $x = \text{vec}(A)$ e $y = \text{vec}(B)$, l'Equazione 1.3 diventa

$$(1.6) \quad \text{vec}(C^\top) = f(x, y) = \mathcal{T} \times_1 \text{vec}(A)^\top \times_2 \text{vec}(B)^\top,$$

dove abbiamo fissato l'ordinamento naturale sulle entrate di $\text{vec}(A)$ e $\text{vec}(B)$, e quello inverso su $\text{vec}(C^\top)$. Da ciò troviamo un tensore $\mathcal{T} \in \mathbb{R}^{MN \times NP \times MP}$ che rappresenta la moltiplicazione tra $\langle M, N, P \rangle$. Data una decomposizione CP di rango r del tensore $\mathcal{T} = \llbracket U, V, W \rrbracket$, grazie all'Equazione 1.5, l'espressione sopra si traduce in

$$(1.7) \quad \text{vec}(C^\top) = \sum_{l=1}^r \underbrace{(u_l^\top \text{vec}(A))}_{s_l} \cdot \underbrace{(v_l^\top \text{vec}(B))}_{t_l} w_l = \sum_{l=1}^r \underbrace{(s_l \cdot t_l)}_{m_l} w_l,$$

Che definisce un algoritmo di moltiplicazione veloce $\langle M, N, P \rangle$.

1.5 ALCUNE STIME SULL'ESPONENTE

Dal ragionamento sopra, $\mathbf{R}(\langle n, n, n \rangle)$ è dato dal numero di active multiplication nella moltiplicazione di due matrici, e quindi ne misura la complessità algoritmica. Alla luce di ciò, possiamo riscrivere ω come

$$(1.8) \quad \omega = \lim_{n \rightarrow +\infty} \log_n(\mathbf{R}(\langle n, n, n \rangle)).$$

FIX errore nella formula

Ad esempio, l'algoritmo di Strassen [17] mostra che $\mathbf{R}(\langle 2, 2, 2 \rangle) \leq 7$, e come dimostrato da Winograd [19], $\mathbf{R}(\langle 2, 2, 2 \rangle) = 7$, ovvero non è possibile moltiplicare due matrici 2×2 con meno di sette moltiplicazioni. Nei casi diversi da $\langle 2, 2, 2 \rangle$ il problema si complica, e attualmente abbiamo alcune stime più lasche: già per matrici 3×3 , sappiamo solo che $19 \leq \mathbf{R}(\langle 3, 3, 3 \rangle) \leq 23$ [4, 11], mentre la migliore stima

astintotica dal basso per il caso di matrici $n \times n$ è $\frac{5}{2}n^2 - 3n \leq \mathbf{R}(\langle n, n, n \rangle)$ [3]. Si veda il capitolo 11 di [12] per ulteriori dettagli e stime.

TRATTARE IL CASO GENERALE

Date due matrici $n^i \times n^i$, possiamo vedere la loro moltiplicazione come la moltiplicazione di matrici $n \times n$ sull'anello delle matrici $n^{i-1} \times n^{i-1}$, suddividendole in blocchi. Questo dice anche che, se $\mathbf{R}(\langle n, n, n \rangle) \leq r$, allora $\mathbf{R}(\langle n^i, n^i, n^i \rangle) \leq r^i$; infatti, combinando le prossime Proposizioni 1.5.2 e 1.5.3, e usando che il rango è invariante per isomorfismo, troviamo

$$\mathbf{R}(\langle n^i, n^i, n^i \rangle) = \mathbf{R}(\langle n, n, n \rangle^{\otimes i}) \leq \mathbf{R}(\langle n, n, n \rangle)^i \leq r^i.$$

Quindi, possiamo applicare ricorsivamente l'algoritmo di moltiplicazione veloce, che è disegnato per matrici $n \times n$, per ottenere la moltiplicazione di matrici $n^i \times n^i$ impiegando solo r^i moltiplicazioni.

Se invece $n > n_0$ sono due interi positivi, dato un algoritmo con un caso base fissato $\langle n_0, n_0, n_0 \rangle$, è possibile ottenerne uno per $\langle n, n, n \rangle$, con $n > n_0$, allargando la matrice aggiungendo zeri fino alla dimensione $n_0^{\lceil \log_{n_0} n \rceil}$. In particolare, si ha che

$$\begin{aligned} \mathbf{R}(\langle n, n, n \rangle) &\leq \mathbf{R}(\langle n_0^{\lceil \log_{n_0} n \rceil}, n_0^{\lceil \log_{n_0} n \rceil}, n_0^{\lceil \log_{n_0} n \rceil} \rangle) \quad \text{dalla 1.5.4,} \\ &\leq \mathbf{R}(\langle n_0, n_0, n_0 \rangle)^{\lceil \log_{n_0} n \rceil} \quad \text{dalle 1.5.2 e 1.5.3,} \\ &\leq r^{\lceil \log_{n_0} n \rceil} \quad \text{per ipotesi } \mathbf{R}(\langle n_0, n_0, n_0 \rangle) \leq r, \\ &\leq r \cdot n^{\log_{n_0} r} \quad \text{usando che } \lceil x \rceil \leq x + 1. \end{aligned}$$

Riassumendo, $\mathbf{R}(\langle n, n, n \rangle) \leq r \cdot n^{\log_{n_0} r}$. Dunque ignorando la costante e dall'Equazione 1.8, $\omega \leq \log_{n_0} r$. Ad esempio, per l'algoritmo di Strassen poniamo $r = 7, n_0 = 2$, da cui $\omega \leq \log_2 7 < 2.81$. Abbiamo dimostrato la seguente:

Proposizione 1.5.1. *Se $\mathbf{R}(\langle n, n, n \rangle) \leq r$ per degli interi positivi n, r , allora $n^\omega \leq r$.*

Riportiamo le proposizioni utilizzate per dimostrare la proposizione sopra, dimostrate in [5], Cap. 14,15.

Proposizione 1.5.2. *Siano e, e', h, h', l, l' degli interi positivi. Allora abbiamo*

$$\langle e, h, l \rangle \otimes \langle e', h', l' \rangle \simeq \langle ee', hh', ll' \rangle,$$

cioè le due forme bilineari sono isomorfe.

Proposizione 1.5.3. *Siamo φ_1, φ_2 due mappe bilineari. Allora $\mathbf{R}(\varphi_1 \otimes \varphi_2) \leq \mathbf{R}(\varphi_1)\mathbf{R}(\varphi_2)$.*

Proposizione 1.5.4 (Monotonia del rango di un algoritmo). *Il rango di un algoritmo è monotono crescente rispetto alla dimensione, ovvero dati due interi positivi $n \leq n'$, allora $\mathbf{R}(\langle n, n, n \rangle) \leq \mathbf{R}(\langle n', n', n' \rangle)$.*

1.6 RANGO COME MISURA DELLA COMPLESSITÀ

La prossima proposizione mostra che il rango di un tensore è una misura ottimale della complessità:

Teorema 1.6.1 (Strassen [17], vedi anche [5] §15.1).

$$\omega = \inf\{\tau \in \mathbb{R} \mid \mathbf{R}(\langle n, n, n \rangle) = O(n^\tau)\}.$$

Ovvero, è possibile moltiplicare due matrici $n \times n$ usando $O(n^{\omega+\varepsilon})$ operazioni aritmetiche se e solo se il rango tensoriale di $\langle n, n, n \rangle$ è $O(n^{\omega+\varepsilon})$.

Dimostrazione. Chiamiamo θ il lato destro dell'equazione sopra. Dall'Equazione 1.7 abbiamo

$$\frac{1}{2}\mathbf{R}(\langle n, n, n \rangle) \leq L(\langle n, n, n \rangle) \leq M(n),$$

va giustificata
meglio (vedi
libro)

calcolando gli inf, troviamo $\theta \leq \omega$.

Mostriamo che $\omega \leq \theta$. Per definizione di θ , per ogni $\varepsilon > 0$ esiste un $m > 1$ tale che

$$r := \mathbf{R}(\langle m, m, m \rangle) \leq m^{\theta+\varepsilon}.$$



□

Espandere di-
scussione dal-
l'articolo

Osservazione 1.6.2. Di seguito ci limiteremo a studiare prodotti fra matrici quadrate; per il caso rettangolare, si possono estendere i ragionamenti sopra considerando una rappresentazione di rango r del tensore $\langle m, n, p \rangle$ e scrivendo, attraverso una serie di permutazioni, la fattorizzazione di $\langle mnp, mnp, mnp \rangle$ di rango r^3 , che produce un algoritmo di complessità $O(n^{\log_{abc} r^3}) = O(n^{3 \log_{abc} r})$

1.7 ALGORITMI DI MOLTIPLICAZIONE VELOCE

IL CASO NAIVE

Trattiamo il caso di matrici quadrate con taglie potenze di due (possiamo farlo a meno di passare a una sottomatrice di taglia pari). In questo caso possiamo partizionare le matrici in delle sottomatrici a blocchi 2×2 ¹:

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}.$$

L'algoritmo naive procede combinando un insieme di otto moltiplicazioni matriciali con quattro addizioni:

$$\begin{aligned} m_1 &= a_{11} \cdot b_{11} & m_2 &= a_{12} \cdot b_{21} & m_3 &= a_{11} \cdot b_{12} \\ m_4 &= a_{12} \cdot b_{22} & m_5 &= a_{21} \cdot b_{11} & m_6 &= a_{22} \cdot b_{21} \\ m_7 &= a_{21} \cdot b_{12} & m_8 &= a_{22} \cdot b_{22} \end{aligned}$$

$$\begin{aligned} c_{11} &= m_1 + m_2 = a_{11}b_{11} + a_{12}b_{21}, & c_{12} &= m_3 + m_4 = a_{11}b_{12} + a_{12}b_{22}, \\ c_{21} &= m_5 + m_6 = a_{21}b_{11} + a_{22}b_{21}, & c_{22} &= m_7 + m_8 = a_{21}b_{12} + a_{22}b_{22}. \end{aligned}$$

Usando ?? possiamo scrivere l'algoritmo $\langle 2, 2, 2 \rangle$ in forma tensoriale in formato CP:

$$\begin{aligned} \langle 2, 2, 2 \rangle &= a_{11} \circ b_{11} \circ c_{11} + a_{12} \circ b_{21} \circ c_{11} \\ &+ a_{21} \circ b_{11} \circ c_{21} + a_{22} \circ b_{21} \circ c_{21} \\ &+ a_{11} \circ b_{12} \circ c_{12} + a_{12} \circ b_{22} \circ c_{12} \\ &+ a_{21} \circ b_{12} \circ c_{22} + a_{22} \circ b_{22} \circ c_{22}. \end{aligned} \tag{1.9}$$

Si vede come si ricava ma non so spiegare come si ottiene

¹possiamo pensare alle entrate sia come scalari, che come matrici.

Per secoli questo algoritmo è stato ritenuto il più veloce per moltiplicare le matrici. Nel ?? Strassen² tentò di mostrare che non esistesse un algoritmo più veloce, fallendo in maniera spettacolare³ e dando vita a una nuova teoria.

L'ALGORITMO DI STRASSEN

L'algoritmo di Strassen è probabilmente l'algoritmo di moltiplicazione veloce più noto e segue la stessa struttura a blocchi $\langle 2, 2, 2 \rangle$.

$$\begin{array}{lll} s_1 = a_{11} + a_{22} & s_2 = a_{21} + a_{22} & s_3 = a_{11} \\ s_4 = a_{22} & s_5 = a_{11} + a_{12} & s_6 = a_{21} - a_{11} \\ s_7 = a_{12} - a_{22} & & \\ t_1 = b_{11} + b_{22} & t_2 = b_{11} & t_3 = b_{12} - b_{22} \\ t_4 = b_{21} - b_{11} & t_5 = b_{22} & t_6 = b_{11} + b_{12} \\ t_7 = b_{21} + b_{22} & & \end{array}$$

$$\begin{array}{ll} m_r = s_r t_r, & 1 \leq r \leq 7 \\ c_{11} = m_1 + m_4 - m_5 + m_7 & c_{12} = m_3 + m_5 \\ c_{21} = m_2 + m_4 & c_{22} = m_1 - m_2 + m_3 + m_6 \end{array}$$

Aggiungere ten-
sore

aggiungere co-
sto

1.8 ALGORITMI APA

Dopo il tentativo di Strassen di dimostrare che l'algoritmo di moltiplicazione standard fosse ottimale, Bini et. Al [2] considerano il seguente algoritmo di moltiplicazione ridotto ottenuto ponendo a zero l'entrata a_{22} di $\langle 2, 2, 2 \rangle$:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & 0 \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} & a_{21}b_{12} \end{bmatrix}.$$

²in una comunicazione riportata da Landsberg in [12].

³in realtà la tesi è vera nel campo $\mathbb{Z}/2\mathbb{Z}$.

Facendo alcuni raccoglimenti e sostituendo $a_{22} = 0$, il tensore dell'Equazione 1.9 si riscrive come

$$\begin{aligned} \langle 2, 2, 2 \rangle^{\text{red}} &:= a_{11} \circ (b_{11} \circ c_{11} + b_{12} \circ c_{12}) + a_{12} \circ (b_{21} \circ c_{11} + b_{22} \circ c_{12}) \\ &\quad + a_{21} \circ (b_{11} \circ c_{21} + b_{12} \circ c_{22}). \end{aligned}$$

Che è somma di sei tensori elementari distinti, quindi $R\langle 2, 2, 2 \rangle^{\text{red}} \leq 6$. Bini et. al. provarono a cercare un'espressione di rango cinque per $\langle 2, 2, 2 \rangle^{\text{red}}$ numericamente, calcolando

$$\min_{\substack{T \in \mathbb{R}^{n \times n \times n} \\ R(T)=5}} \|\langle 2, 2, 2 \rangle^{\text{red}} - T\|$$

norma di $\langle 2, 2, 2 \rangle^{\text{red}}$. Il loro computer continuava a produrre tensori T di rango cinque con la norma della differenza che diventava sempre più piccola, ma con coefficienti sempre più grandi. Questo è andato avanti per un po' di tempo, fino a che realizzarono che il codice era giusto, ma che era una manifestazione di un fenomeno chiamato *border rank*. L'espressione del tensore che stavano cercando era essenzialmente

finire

IL PROBLEMA DEL BORDER RANK

Esempio 1.8.1. Consideriamo dei vettori $a_1, a_2 \in \mathbb{R}^m$, $b_1, b_2 \in \mathbb{R}^n$ e $c_1, c_2 \in \mathbb{R}^p$ vettori linearmente indipendenti. Definiamo il tensore

$$(1.10) \quad \mathcal{X} = a_1 \circ b_1 \circ c_2 + a_1 \circ b_2 \circ c_1 + a_2 \circ b_1 \circ c_1,$$

e notiamo che $R(\mathcal{X}) = 3$ dato che non può essere scomposto come somma di tensori più semplici. Consideriamo la successioni di tensori

$$(1.11) \quad \mathcal{X}_\varepsilon = \frac{1}{\varepsilon}(a_1 + \varepsilon a_2) \circ (b_1 + \varepsilon b_2) \circ (c_1 + \varepsilon c_2) - \frac{1}{\varepsilon}a_1 \circ b_1 \circ c_1, \text{ con } \varepsilon \in \mathbb{R},$$

che essendo combinazione lineare di due tensori di rango uno, ha rango 2. Vediamo che $\lim_{\varepsilon \rightarrow 0} \mathcal{X}_\varepsilon = \mathcal{X}$, da cui segue che l'insieme dei tensori di rango 3 non è chiu-

Introduci e motiva il problema partendo dagli algoritmi APA (vedi Landsbeg)

so. Espandendo i termini dell'Equazione 1.11 tramite la proprietà distributiva del prodotto esterno,

$$\begin{aligned}
 (1.12) \quad \mathcal{Y}_\varepsilon &= \frac{1}{\varepsilon} a_1 \circ b_1 \circ c_2 + a_1 \circ b_1 \circ c_2 + a_1 \circ b_2 \circ c_1 + a_2 \circ b_1 \circ c_1 \\
 &\quad + \varepsilon(a_2 \circ b_2 \circ c_1 + a_1 \circ b_1 \circ c_2 + a_1 \circ b_2 \circ c_2 + a_2 \circ b_2 \circ c_2) - \frac{1}{\varepsilon} a_1 \circ b_1 \circ c_1 \\
 &= \mathcal{X} + \varepsilon(a_2 \circ b_2 \circ c_1 + a_1 \circ b_1 \circ c_2 + a_1 \circ b_2 \circ c_2 + a_2 \circ b_2 \circ c_2)
 \end{aligned}$$

Quindi $\mathcal{X}_\varepsilon - \mathcal{X} \rightarrow 0$.

Questo motiva la seguente definizione:

Definizione 1.8.2 (Border rank). Dato un tensore $\mathcal{X} \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$, il suo border rank è la quantità

$$\underline{R}(\mathcal{X}) = \min \{ r \in \mathbb{N} \mid \exists \mathcal{Y} \in \mathbb{R}^{n_1 \times \dots \times n_d} \text{ t.c. } R(\mathcal{Y}) = r, \|\mathcal{X} - \mathcal{Y}\| < \varepsilon \}.$$

Alternativa:

Definizione 1.8.3 (Border rank). Un tensore $\mathcal{X}$ ha *border rank* r se è limite di una successione di tensori di rango r , ma non è limite di una successione di tensori di rango s , per ogni $s < r$. Lo denoteremo con il simbolo $\underline{R}(\mathcal{X})$.

Osservazione 1.8.4. $R(\mathcal{X}) \geq \underline{R}(\mathcal{X})$.

1.9 ERRORE ALGORITMICO

Disegno interpretazione geometrica (landsberg pg 38)

Alcune stime su border rank di matrix mult operator

discussione della stabilità dal paper dato dal prof

2 APPLICAZIONI DELLA CP ALLA ??

Presentiamo alcune applicazioni della CP in altre discipline.

2.1 FLUORESCENZA SPECTORSOPICA

2.2 COCKTAIL PARTY PROBLEM

Un problema centrale nel signal processing è il *problema del cocktail party*, dove un insieme di persone si trova in una stanza a parlare. Ci sono diversi ricevitori impostati che registrano le conversazioni: non singolarmente ma tutte insieme, sovrapposte. L'obiettivo è recuperare cosa ogni persona sta dicendo. Questo "unmixing" si può ottenere usando la decomposizione CP.

Explain

3 APPENDICE

3.1 IL MASTER METHOD

Introduciamo brevemente un metodo per risolvere relazioni ricorsive definite per ricorrenza, detto *master method*. I lettori più interessati possono trovare una trattazione più dettagliata sul libro [7]. Il master method si usa per risolvere ricorrenze della forma

$$(3.1) \quad T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

dove $a \geq 1$ e $b > 1$ sono costanti e $f(n)$ è una funzione asintoticamente positiva. La ricorrenza 3.1 descrive il tempo di esecuzione di un algoritmo che divide un problema di dimensione n in a sottoproblemi, ognuno di dimensione $\frac{n}{b}$. Gli a sottoproblemi vengono risolti ricorsivamente in tempo $T(\frac{n}{b})$, e la funzione $f(n)$ rappresenta il costo di dividere il problema e ricombinare la ricorsione di ogni sottoproblema.

Definizione 3.1.1 (Notazione O –grande/piccolo). Per delle funzioni f, g a variabile reale (o intera) nella variabile x diciamo che:

- $f(x) = O(g(x))$ se esiste una costante $C > 0$ e un valore x_0 tale che $|f(x)| \leq C|g(x)|$ per ogni $x \geq x_0$.
- $f(x) = o(g(x))$ se $\lim_{x \rightarrow \infty} \frac{|f(x)|}{|g(x)|} = 0$.
- $f(x) = \Omega(g(x))$ se esiste una costante $C > 0$ e un valore x_0 tale che $C|f(x)| \geq |g(x)|$ per ogni $x \geq x_0$.
- $f(x) = \omega(g(x))$ se $\lim_{x \rightarrow \infty} \frac{|g(x)|}{|f(x)|} = 0$.
- $f(x) = \Theta(g(x))$ se $f(x) = O(g(x))$ e $f(x) = \Omega(g(x))$.

Teorema 3.1.2 (Master theorem). Siano $a \geq 1$ e $b > 1$ delle costanti, sia $f(n)$ una funzione e definiamo la legge ricorsiva $T : \mathbb{N} \rightarrow \mathbb{R}$ con la ricorrenza

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

3 Appendice

dove con $\frac{n}{b}$ indichiamo sia $\lfloor \frac{n}{b} \rfloor$ o $\lceil \frac{n}{b} \rceil$. Allora $T(n)$ assume i seguenti comportamenti asintotici:

1. Se $f(n) = O(n^{\log_b a - \varepsilon})$ per qualche $\varepsilon > 0$, allora $T(n) = \Theta(n^{\log_b a})$.
2. Se $f(n) = \Theta(n^{\log_b a})$ allora $T(n) = \Theta(n^{\log_b a \log n})$.
3. If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ per qualche costante $\varepsilon > 0$, e se $af(\frac{n}{b}) < cf(n)$ per qualche costante $c < 1$ e ogni n sufficientemente grande, allora $T(n) = \Theta(f(n))$.

Esempio 3.1.3. Nel caso dell'algoritmo naive la ricorrenza ha la forma $a = 8$, $b = 2$ e $f(n) = \Theta(n^2)$, ovvero

$$T(n) = 8T\left(\frac{n}{2}\right) + \Theta(n^2).$$

Siccome $n^{\log_b a} = n^{\log_2 8} = n^3$, che è polinomialmente più grande di $f(n)$, ovvero $f(n) = O(n^{3-\varepsilon})$ per $\varepsilon = 1$, dal caso 1 di 3.1.2 segue che $T(n) = \Theta(n^3)$. Nel caso di Strassen, si ripetono i calcoli con $a = 7$, $b = 2$ e $f(n) = \Theta(n^2)$ e si ottiene la relazione $T(n) = 7T(\frac{n}{2}) + \Theta(n^2)$ e analogamente $T(n) = \Theta(n^{\log_2 7})$.

ACRONIMI

PCA	Principal component analysis
SNF	Smith normal form
TDA	Topological data analysis

GLOSSARIO

$\text{\LaTeX}$	A document preparation system
$\mathbb{R}$	The set of real numbers

BIBLIOGRAFIA

1. J. Alman, R. Duan, V. V. Williams, Y. Xu, Z. Xu, e R. Zhou. «More asymmetry yields faster matrix multiplication». *Computing Research Repository (arXiv)*, 2024. arXiv:2404.16349.
2. D. Bini. «Relations between exact and approximate bilinear algorithms. Applications». *Calcolo* 17:1, 1980, pp. 87–97. DOI: [10.1007/bf02575865](https://doi.org/10.1007/bf02575865). URL: <https://doi.org/10.1007/bf02575865>.
3. M. Bläser. «A $(5/2)n^2$ -Lower Bound for the Multiplicative Complexity of $n \times n$ -Matrix Multiplication». In: *Proceedings of the 18th Annual Symposium on Theoretical Aspects of Computer Science*. STACS '01. Springer-Verlag, Berlin, Heidelberg, 2001, pp. 99–109. ISBN: 3540416951.
4. M. Bläser. «On the complexity of the multiplication of matrices of small formats». *Journal of Complexity* 19:1, 2003, pp. 43–60. ISSN: 0885-064X. DOI: [https://doi.org/10.1016/S0885-064X\(02\)00007-9](https://doi.org/10.1016/S0885-064X(02)00007-9). URL: <https://www.sciencedirect.com/science/article/pii/S0885064X02000079>.
5. P. Bürgisser, M. Clausen, e M. A. Shokrollahi. *Algebraic complexity theory*. Vol. 315. Springer Science & Business Media, 2013.
6. D. Coppersmith e S. Winograd. «Matrix multiplication via arithmetic progressions». *Journal of Symbolic Computation* 9:3, 1990. Computational algebraic complexity editorial, pp. 251–280. ISSN: 0747-7171. DOI: [https://doi.org/10.1016/S0747-7171\(08\)80013-2](https://doi.org/10.1016/S0747-7171(08)80013-2). URL: <https://www.sciencedirect.com/science/article/pii/S0747717108800132>.
7. T. H. Cormen, C. E. Leiserson, R. L. Rivest, e C. Stein. *Introduction to algorithms*. MIT press, 2022.
8. P. D’Alberto. *Strassen’s Matrix Multiplication Algorithm Is Still Faster*. 2023. arXiv: [2312.12732](https://arxiv.org/abs/2312.12732) [cs.MS]. URL: <https://arxiv.org/abs/2312.12732>.

9. A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatin, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz et al. «Discovering faster matrix multiplication algorithms with reinforcement learning». *Nature* 610:7930, 2022, pp. 47–53.
10. F. L. Gall. «Powers of Tensors and Fast Matrix Multiplication». *CoRR* abs/1401.7714, 2014. arXiv: [1401.7714](https://arxiv.org/abs/1401.7714). URL: <http://arxiv.org/abs/1401.7714>.
11. J. D. Laderman. «A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications». *Bulletin of the American Mathematical Society* 82:1, 1976, pp. 126–128.
12. J. M. Landsberg. *Tensors: geometry and applications*. Vol. 128. American Mathematical Soc., 2011.
13. V. Y. Pan. «Strassen’s algorithm is not optimal trilinear technique of aggregating, uniting and canceling for constructing fast algorithms for matrix operations». In: *19th Annual Symposium on Foundations of Computer Science (sfcs 1978)*. 1978, pp. 166–176. DOI: [10.1109/SFCS.1978.34](https://doi.org/10.1109/SFCS.1978.34).
14. A. Schönhage. «Partial and Total Matrix Multiplication». *SIAM Journal on Computing* 10:3, 1981, pp. 434–455. DOI: [10.1137/0210032](https://doi.org/10.1137/0210032). eprint: <https://doi.org/10.1137/0210032>. URL: <https://doi.org/10.1137/0210032>.
15. A. Stothers. «On the complexity of matrix multiplication». Tesi di dott. Edinburgh, UK: University of Edinburgh, 2010. URL: <https://ed.ac.uk>.
16. V. STRASSEN. «Relative bilinear complexity and matrix multiplication.» *Journal für die reine und angewandte Mathematik* 1987:375-376, 1987, pp. 406–443. DOI: [doi:10.1515/crll.1987.375-376.406](https://doi.org/10.1515/crll.1987.375-376.406). URL: <https://doi.org/10.1515/crll.1987.375-376.406>.
17. V. Strassen. «Gaussian elimination is not optimal». *Numer. Math.* 13:4, 1969, pp. 354–356. ISSN: 0029-599X. DOI: [10.1007/BF02165411](https://doi.org/10.1007/BF02165411). URL: <https://doi.org/10.1007/BF02165411>.
18. V. V. Williams. *Breaking the Coppersmith-Winograd barrier*. Manuscript available at <http://theory.stanford.edu/~virgi/matrixmult-f.pdf>. 2011.

19. S. Winograd. «On multiplication of 2×2 matrices». *Linear Algebra and its Applications* 4:4, 1971, pp. 381–388. ISSN: 0024-3795. DOI: [https://doi.org/10.1016/0024-3795\(71\)90009-7](https://doi.org/10.1016/0024-3795(71)90009-7). URL: <https://www.sciencedirect.com/science/article/pii/0024379571900097>.